

Python Syntax Review

CS 101 Algorithmic Problem Solving

Fall 2024

This week is the first time you will be asked to write code in this course; albeit on paper. This reading serves as a review of the python syntax of everything you've learned up until this point. You might also learn some new syntax for statements that you will be using quite often moving forward.

Week 1 Review

In week 1, you learned about variables, expressions, statements, and different python data types. Let's make some variables.

```
a = '123'
b = 123
c = 1.23
```

The variable `a` is of type *string*, or `str` in python, as it is enclosed in quotation marks. The variable `b` is an *integer*, or `int` in python, and the variable `c` is a `float`. You cannot perform arithmetic operations on `a`, even though it looks like it contains a number. However, you can perform all arithmetic operations on `b` and `c`. You can also concatenate `a` with itself or with some other string.

```
>>> print(b/c)
100.0
>>> print(a + a)
123123
>>> print(a + '456')
123456
```

Week 2 Review

In week 2, you learned about conditionals. Giving your program instructions what to do when a certain condition is met and, optionally, what to do if not. Representing conditionals begins with an `if` statement, followed by its associated block which is *indented* by a single tab space as shown in the examples below.

```
if b == 123:
    print('Yay!')
else:
    print('The value of b is', b)
```

The above example has an `if` statement which checks if the value of `b` is 123 and if it is, it prints "Yay!"; otherwise, it prints a statement telling the value of `b` is. In our case, the value of `b` is

123, so it will never enter the else block. Keep in mind how the blocks are indented; python is an indent-sensitive language so it is important to note where to use indentation and what it means. In this example, the indentation is used to denote what lines of code are part of the conditional blocks.

Let's now take a look at nested conditionals.

```
if b == 123:
    print('Yay!')
    if c == 1.23:
        print('Double Yay!')
        if a == '124':
            print('Triple Yay!')
        else:
            print('Only double yay ;-;')
    else:
        print('Only single yay ;-;')
else:
    print('No yay ;-;')
```

The above example contains several nested conditionals. The program first checks if the value of `b` is 123; if it is then it moves to line 2, otherwise it moves to line 12. With our values for the variables, it will move to line 2, execute the `print` statement, move to line 3, enter the conditional block at line 4, execute the `print` statement, move to line 5, the `if` statement is not satisfied so it moves to line 7, enters the `else` block at line 8, and finally executes the `print` statement before ending. Our output would be as follows:

```
Yay!
Double Yay!
Only double yay!
```

Note this might be confusing to read at first, but try to fully understand the flow of this program, and how it would function with different values of `a`, `b`, and `c`.

Doing more with `print`

Earlier in the reading, the keen eyed among you may have noticed a `print` statement that had two arguments. It was in the first example of the Week 2 Review section:

```
print('The value of b is', b)
```

With the `print` statement, if you want to print more than 1 thing in the same statement, you can use a comma to separate them. You can add as many arguments as you want and they can be variables or *literal* values. The above statement will output the following:

```
>>> print('The value of b is', b)
The value of b is 123
```

As you see, it has placed a space character where the comma used to be. That's the default *separator* between the arguments for the `print` statement. **Any spaces around the separator are for aesthetic purposes only and ignore by python.** The `print` statement also by default ends with the `\n` character. This is to say that if you have two `print` statements in the program, one after another, their outputs will be printed on separate lines. Consider the following program:

```
print('Hi!', 'My name is Algo.')
print('How are you?')
```

Running the program would produce the following output:

```
Hi! My name is Algo.  
How are you?
```

As you can see, the two printed statements are on separate lines. It is important to keep these behaviors in mind when you are writing your code. If you wish to learn how to change these default separator and end behaviors, you can check out [this link](#).

Moving forward

Now that we've reviewed what you have learned and also learned a bit more about the `print` statement, let's use all that to make a very simple algorithm.

Suppose you're getting two integers as input, `X` and `Y` and want to make an algorithm which, if the sum of `X` and `Y` is divisible by 5, prints a statement saying "Divisible by 5!"; otherwise, if the sum is divisible by 4, it prints "Divisible by 4!", and if neither condition is met, it prints "The numbers `X` and `Y` are divisible by neither 5 nor 4!".

```
total = X + Y # store the sum of X and Y  
  
if total % 5 == 0: # check if the total is divisible by 5  
    print('Divisible by 5!')  
  
elif total % 4 == 0: # check if the total is divisible by 4  
    print('Divisible by 4!')  
  
else: # neither condition is met  
    print('The numbers', X, 'and', Y, 'are divisible by neither 5 nor 4!')
```

This is the code for the above described algorithm! Now let's see how it works for some values of `X` and `Y`.

If `(X,Y)=(5,5)`, the program will calculate the total which is 10, move to the first conditional on line 3, enter the conditional block on line 4, and print `Divisible by 5!`.

If `(X,Y)=(5,3)`, the program will calculate the total which is 8, move to the first conditional on line 3, move to the second conditional on line 6 as the first condition isn't satisfied, enter the conditional block on line 7, and print `Divisible by 4!`.

If `(X,Y)=(5,2)`, the program will calculate the total which is 7, move to the first conditional on line 3, move to the second conditional on line 6 as the first condition isn't satisfied, enter the `else` block on line 10 as the second condition isn't satisfied, and print `The numbers 5 and 2 are divisible by neither 5 nor 4!`.

It is imperative you understand the flow of your algorithm even without seeing the outputs on a computer screen. Getting proficient at *dry running* your code will prove invaluable throughout your programming journey.